A Mini Project report on

FreshGuard: Freshness-Aware Adaptive Retrieval-Augmented Generation with Statement-Level Hallucination Detection

Mini Project submitted to Anurag University in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Artificial Intelligence

Submitted by

J. Sai Teja	**23EG106C22**
P. Supriya	**23EG106C52**
Sk. Mubeena	**23EG106C58**
U. Venkatesh	**23EG106C63**

Under the Guidance of

[Guide Name]

[Designation]

Department of Artificial Intelligence
School of Engineering
ANURAG UNIVERSITY

2023–2027

ANURAG UNIVERSITY | SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE
CERTIFICATE

This is to certify that the project report titled **FreshGuard: Freshness-Aware Adaptive Retrieval-Augmented Generation with Statement-Level Hallucination Detection** is being submitted by **J. Sai Teja**, bearing **23EG106C22**, **P. Supriya**, bearing **23EG106C52**, **Sk. Mubeena**, bearing **23EG106C58**, and **U. Venkatesh**, bearing **23EG106C63**, in IV B.Tech I semester Artificial Intelligence, and is a record of bonafide work carried out by them. The results embodied in this report have not been submitted to any other University for the award of any degree.

Student's Name	**Signature**
1. J. Sai Teja	1.
2. P. Supriya	2.
3. Sk. Mubeena	3.
4. U. Venkatesh	4.

[Guide Name]	**Dr. M. Trupti**	**External Examiner**
[Designation]	Professor	
Guide	Head of the Department	

Acknowledgement

We are grateful to Prof. Archana Mantri, Vice-Chancellor, Anurag University, for the facilities the University made available to us and for the environment in which this project could be carried out.

We thank Dr. V. Vijay Kumar, Dean, School of Engineering, for his encouragement throughout our course of study.

We sincerely thank Dr. M. Trupti, Professor and Head of the Department of Artificial Intelligence, for providing the resources and support we needed to complete this work.

We are especially indebted to our guide, [Guide Name], [Designation], Department of Artificial Intelligence, Anurag University. Their questions during the reviews shaped how we tested the system, and their feedback on each draft improved this report.

We also thank the developers of the open-source libraries this project is built on, in particular scikit-learn, LangChain, FAISS, FastAPI and Uvicorn.

An AI assistant was used during this project to help with software development and with drafting and editing parts of this report. All experiments were run by us, and every result reported here was checked against the logged output of the system.

CONTENTS

(In Microsoft Word, right-click the table below and choose "Update Field" to populate page numbers.)

ABSTRACT

Large language models answer open-domain questions fluently, but their knowledge is frozen at training time. For time-sensitive questions they state facts that are simply out of date, and even retrieval-augmented systems rarely check the answer the model finally writes. In practice an out-of-date or unsupported answer is worse than no answer, because the reader may act on it.

This project builds FreshGuard, a freshness-aware retrieval-augmented generation (RAG) system in which retrieval is conditional and every answer is verified before the user sees it. A learned knowledge-freshness estimator scores how time-sensitive a question is and drives an explicit retrieve-or-skip decision, so the system retrieves fresh evidence only when its own knowledge is likely stale and answers directly otherwise. When it retrieves, a LangChain FAISS index ranks evidence by relevance and recency, the language model must ground its answer in that evidence, and a statement-level verifier labels every sentence supported, unsupported or contradicted. Unsupported sentences are removed and contradicted ones are flagged, and the checks are reduced to a single 0–100 confidence score shown with cited sources.

On a controlled benchmark, the freshness gate classified all twelve held-out queries correctly, the statement-level verifier caught every injected hallucination (precision, recall and F1 of 1.00 on eight supported/corrupted pairs), factuality was 1.00 on the answerable questions, and the expected calibration error was 0.10. The system is fully offline and deterministic by default; its 42-test suite reproduced identically across twelve consecutive runs. The main limitation is the small, mostly-correct evaluation set, which makes threshold calibration on harder data the most important next step.

Keywords: Retrieval-Augmented Generation, Knowledge Freshness, Adaptive Retrieval, Hallucination Detection, Statement Verification, Calibration, FAISS, LangChain.

LIST OF FIGURES

Figure 1.1 High-level block diagram of FreshGuard: the freshness gate, conditional retrieval, grounded generation and the verifier.

Figure 3.1 System architecture: web interface, FastAPI backend, pipeline stages, the learned estimator, the FAISS store and SQLite.

Figure 3.2 The adaptive decision: the freshness gate routes each query to retrieval-plus-verification or to a direct answer.

Figure 6.1 Ask page: a verified answer with its confidence, freshness badge, per-statement verification and cited source.

Figure 6.2 Freshness scores for static and time-sensitive queries, with the decision threshold.

Figure 6.3 Evaluation summary: freshness accuracy, factuality and hallucination precision, recall and F1.

Figure 6.4 Mean latency per query on the skip and retrieve paths.

LIST OF TABLES

Table 2.1 Summary of related work and how this project builds on it.

Table 3.1 Knowledge base and query data used in the project.

Table 4.1 Modules of the system.

Table 5.1 Automated test suites.

Table 6.1 Controlled benchmark results.

Table 6.2 Latency per query by path.

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

An answer is only useful when it is both correct and current. Whether a question is about who holds a title this year, what an interest rate is now, or how large a device's memory has become, the right answer changes over time, and a reader who relies on a stale answer is misled as surely as by a wrong one.

Large language models (LLMs) such as GPT, Gemini, Claude and Grok have made it easy to ask such questions in plain language, and their answers are fluent and confident. The difficulty is that an LLM's knowledge is frozen at its training cutoff. For a timeless question—the speed of light, the capital of a country—this is no problem, but for a time-sensitive one the model answers from a world that may be months or years out of date, with no signal that anything has changed. Fluency is not currency, and it is not accuracy.

Retrieval-augmented generation (RAG) [1] is the standard remedy: instead of answering from memory, the model is given passages retrieved from a trusted collection and asked to answer from them. RAG improves grounding, but two problems survive it. Retrieval is usually unconditional, so even questions the model already knows trigger a search, which wastes work and can inject irrelevant or conflicting passages. And nothing in a basic RAG system checks the generated answer against the evidence before a user reads it: the model can still misread a passage, combine two incorrectly, or state something the evidence does not support.

The motivation for this project was therefore not to build another chatbot, but a system that decides when fresh evidence is actually needed and that treats verification as a required step: an answer should reach the user only after each of its sentences has been checked against the evidence, and when a sentence is not supported it should be removed rather than shown.

1.2 PROBLEM STATEMENT

The project addresses the following problems:

1. **Stale parametric answers:** LLMs answer time-sensitive questions from training-time knowledge, with no indication that the answer may be out of date.
2. **Unconditional retrieval:** RAG systems retrieve for every question, including timeless ones the model already answers correctly, paying the cost and the noise of retrieval needlessly.
3. **Unverified answers:** RAG systems show the model's answer as written, even when a sentence is not supported by the retrieved text or contradicts it.
4. **No evidence for the reader:** users are rarely shown which passage supports which sentence, so they cannot check an answer without reading the whole source.

Hence there is a need for a question-answering system that estimates when knowledge is likely stale, retrieves only then, grounds its answer in recency-ranked evidence, checks every sentence against that evidence before display, and reports a calibrated confidence with citations.

1.3 SCOPE AND APPLICATIONS

The system answers questions in English over a knowledge base of dated documents. It is a research prototype and does not give professional advice. Possible uses include:

- **Current-affairs and factual assistants:** answering questions where the correct answer changes over time, while clearly skipping retrieval for timeless ones.
- **Enterprise knowledge bases:** question answering over dated internal documents, where recency matters and every answer must be traceable to a source.
- **Trustworthy-AI research:** the freshness gate, the statement-level verifier and the logged evaluation can be reused to study adaptive retrieval and hallucination detection.
- **Teaching:** a small, fully offline and deterministic RAG system that students can read end to end and run without any API key.

1.4 CONTRIBUTIONS OF THE PROJECT

5. A learned knowledge-freshness estimator that scores how time-sensitive a question is and drives an explicit retrieve-or-skip decision, rather than a fixed time-decay rule.
6. A recency-aware retrieval stage over a LangChain FAISS index that ranks evidence by both relevance and age, with a transformer encoder and a live web source as pluggable seams.
7. A statement-level verification chain that labels each sentence supported, unsupported or contradicted (with entity-swap and numeric-mismatch detection), removes or flags bad sentences, and reduces the checks to one calibrated confidence score with citations.
8. A fully offline, deterministic implementation with a measured evaluation of freshness, factuality, hallucination detection, calibration and latency, and a usable web application.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews existing work and identifies the gaps this project addresses. Chapter 3 describes the proposed system and its requirements, and Chapter 4 its design through data-flow and module descriptions. Chapter 5 covers implementation and testing, Chapter 6 presents the outputs and results, and Chapters 7 and 8 give the conclusion and future work. The annexure explains how to install and run the system and lists the API.

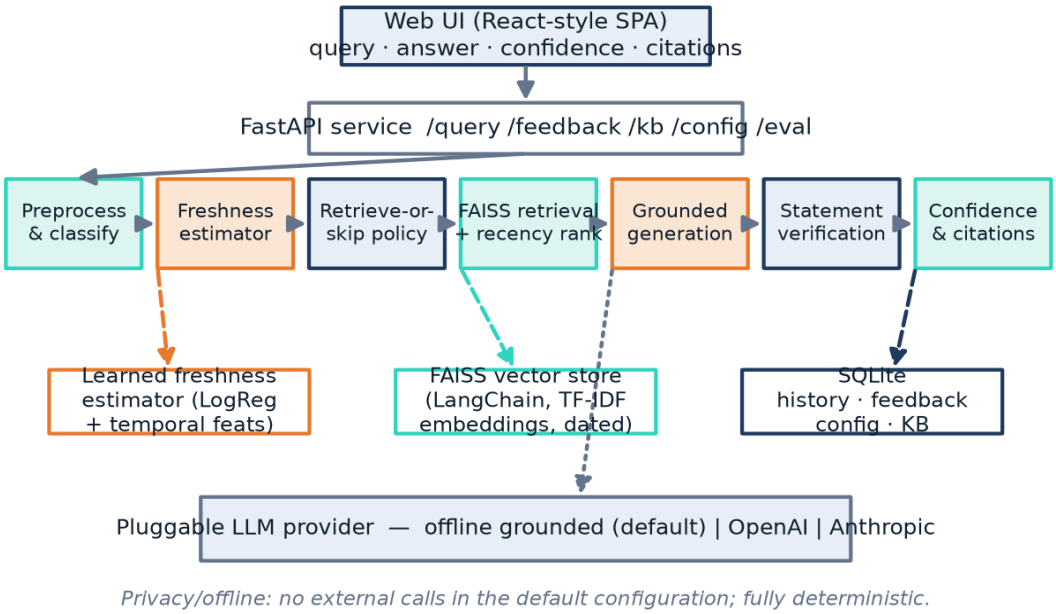


Figure 1.1: High-level block diagram of FreshGuard: the freshness gate, conditional retrieval, grounded generation and the verifier.

CHAPTER 2: LITERATURE SURVEY

2.1 EXISTING SYSTEM

Retrieval-augmented generation. Lewis et al. [1] introduced RAG, in which a retriever selects passages from a collection and a generator conditions its answer on them. Dense passage retrieval [2] and efficient similarity search with FAISS [3] made this practical at scale, and most current assistants follow the design: chunk, embed, retrieve the nearest passages, and generate.

Knowledge freshness. RealTimeQA [9] and FreshLLMs [10] show how quickly a static model's answers age on time-sensitive questions and argue for search-augmented answering. PopQA [11] shows that models should lean on parametric memory for popular facts and on retrieval for rare or changing ones—exactly the distinction a freshness gate must make.

Adaptive retrieval. Self-RAG [7] trains a model to decide when to retrieve and to critique its own output, and FLARE [8] retrieves on demand during generation. Both make retrieval conditional but judge the answer as a whole rather than sentence by sentence.

Hallucination detection. The hallucination survey of Ji et al. [4] frames the problem. SelfCheckGPT [5] samples several answers and treats facts that do not recur as likely hallucinations, and FActScore [6] breaks a long answer into atomic facts and checks each one. Calibration of confidence is studied by Guo et al. [12]. Classic keyword retrieval (BM25 [13]) remains a strong complement to dense search.

Frameworks. scikit-learn [14] provides the learned estimator, and LangChain [15] provides the retrieval and vector-store abstractions used here over a FAISS index.

Table 2.1: Summary of related work and how this project builds on it.

Work	Approach	Limitation	In this project
Lewis et al. [1]	RAG: retrieve then generate	No check on the final answer; always retrieves	Conditional retrieval; every sentence verified
FreshLLMs [10], RealTimeQA [9]	Measure/avoid staleness with search	Benchmark or always-search	A learned gate decides when to retrieve
Self-RAG [7], FLARE [8]	Adaptive, on-demand retrieval	Judge the whole answer	Statement-level labels and gating
SelfCheckGPT [5]	Consistency across samples	Extra model calls	Coverage + contradiction rules, offline
FActScore [6]	Atomic-fact checking	Evaluation tool, not a gate	Used as the basis of the verifier

2.2 LIMITATIONS OF EXISTING SYSTEM

- **Stale answers shown with confidence.** A static LLM gives no signal that a time-sensitive answer is out of date.
- **Retrieval is unconditional.** Every query triggers a search, wasting work and risking noise on questions the model already knows.

- **Answers are shown unchecked.** Basic RAG trusts the model to stay faithful; nothing prevents an unsupported sentence from reaching the user.
- **Little transparency.** Users are rarely shown the quoted evidence and the verdict for each sentence, so they must trust the answer or read the whole source.
- **Heavy, non-deterministic infrastructure.** Many systems need a hosted model and a separate vector server and cannot be reproduced exactly.

2.3 GAPS IDENTIFIED

- A learned gate that decides, per question, whether fresh evidence is needed, rather than a fixed decay rule or always-retrieve.
- Recency as a first-class ranking signal, so a fresher document outranks an equally relevant older one.
- Per-sentence verification that labels and acts on each statement—removing unsupported ones and flagging contradictions—combined into one calibrated score.
- A fully offline, deterministic reference implementation that can be read, run and reproduced without any API key.

2.4 PROBLEM STATEMENT

Existing RAG systems ground answers in retrieved documents but retrieve unconditionally, do not rank evidence by recency, do not verify the answer sentence by sentence, and rarely show the user the evidence behind each statement. This project aims to design and implement a question-answering system that (1) estimates how time-sensitive a question is with a learned model and retrieves only when needed, (2) ranks evidence by relevance and recency over a FAISS index, (3) grounds the answer in that evidence and checks every sentence against it, removing unsupported sentences and flagging contradictions, and (4) reports a calibrated confidence with citations.

2.5 OBJECTIVES

- Build a preprocessing and classification stage for natural-language queries.
- Train a learned knowledge-freshness estimator and implement the retrieve-or-skip decision.
- Implement recency-aware retrieval over a LangChain FAISS index, with pluggable embeddings and a web-search seam.
- Implement grounded generation behind a provider adapter (offline grounded by default; OpenAI, Anthropic and Grok optional).
- Design and implement the statement-level verifier, the corrective action and the confidence score.

- Provide a web interface, and evaluate freshness, factuality, hallucination detection, calibration and latency, reporting the results honestly, including limitations.

CHAPTER 3: PROPOSED SYSTEM

3.1 ARCHITECTURE / ALGORITHMS / METHODS

FreshGuard is organised as a web application on top of a question-answering pipeline (Figure 3.1). A single-page interface talks to a FastAPI backend over HTTP; the backend runs the pipeline stages, each a separate module, and stores data in an embedded SQLite database and a FAISS index on disk. The language model is called through a provider adapter, while the freshness estimator and the embeddings run locally.

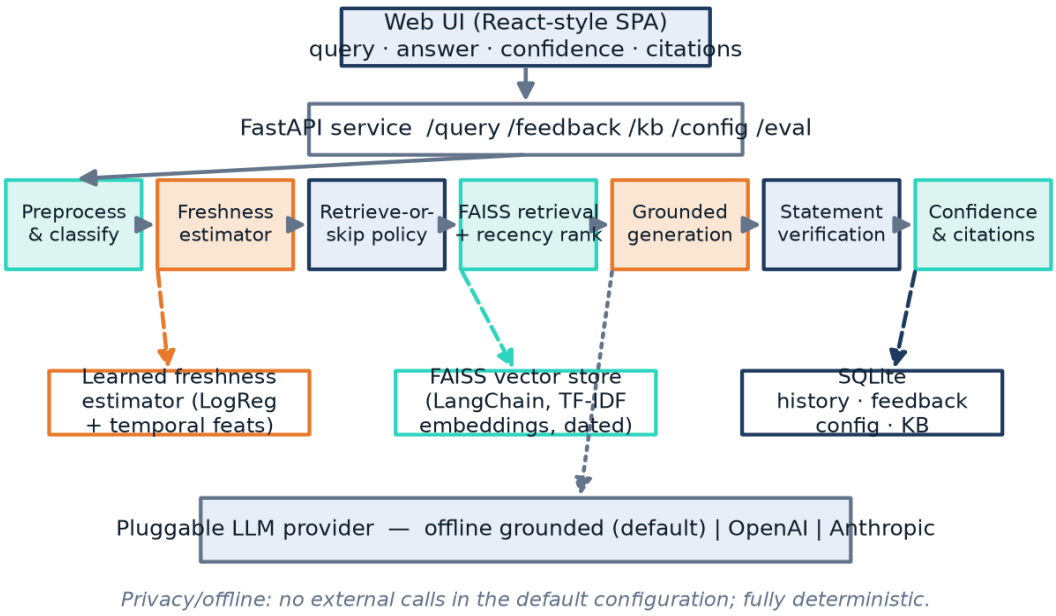


Figure 3.1: System architecture: web interface, FastAPI backend, pipeline stages, the learned estimator, the FAISS store and SQLite.

3.1.1 User Interface (Front-end)

The front-end is a single-page application (HTML, CSS and vanilla JavaScript) with three tabs. Ask provides a query box with example chips and shows each answer with a confidence meter, a freshness/retrieval/category badge, the grounded answer, any warnings, a per-statement verification list and the cited sources (title, URL, date, relevance and recency), together with star rating and “report incorrect” feedback. History lists past queries and lets the user reopen any answer. Admin edits the configuration (thresholds, top-k, provider), manages the knowledge base (add, list, delete) and runs the evaluation.

3.1.2 Backend Server (FastAPI)

The backend is a FastAPI application exposing endpoints to answer a question through the full pipeline, submit feedback, read history, read and update configuration, manage the knowledge base and run the evaluation (Annexure A.3). Pydantic models validate every request and

response, and each failure is returned with its real reason rather than a generic error, so the interface and the tests can rely on the status codes.

3.1.3 Preprocessing and Classification

A query is first cleaned—whitespace normalised and case folded for matching—and classified into a topic bucket (sports, finance, technology, science, geography, politics) by keyword. A simple time-sensitivity flag is also computed from explicit cues. The topic is shown to the user and stored with the history; the learned estimator, not these keywords, makes the retrieval decision.

3.1.4 Learned Freshness Estimator

The core of the system is a classifier that scores how much a question depends on fresh knowledge. It is a scikit-learn pipeline whose features are a union of word TF-IDF, character n-gram (char_wb) TF-IDF and six explicit temporal features—counts of temporal, event and static cue words, the number of year mentions, whether a recent year (≥ 2020) appears, and the query length—fed to a logistic regression [14]. It is trained once on a synthesised, labelled query set (375 queries from "fresh" and "static" templates, including time-sensitive phrasings that use only an explicit year with no "latest/current" cue) and persisted to disk. The freshness score s is the model's probability that the query is time-sensitive, and a self-confidence c is read from the same probability. Because the estimator is learned rather than rule-based, it generalises to phrasings a keyword rule would miss, such as "who won the 2022 world cup".

3.1.5 Retrieve-or-Skip Decision

With freshness score s, self-confidence c, freshness threshold $\tau = 0.55$ and confidence threshold $\tau c = 0.60$, the system retrieves if and only if $s \geq \tau$ or $c < \tau c$ (Equation 3.1). A question is answered directly only when it looks timeless and the model is confident; otherwise the system retrieves. This is the first of FreshGuard's two gates.

$$retrieve \iff (s \geq \tau) \vee (c < \tau c) \qquad (3.1)$$

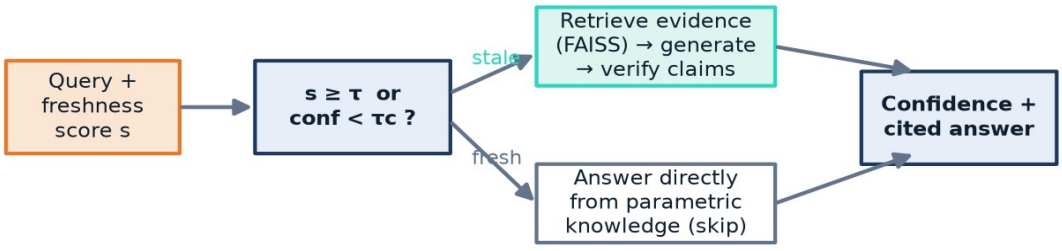


Figure 3.2: The adaptive decision. The freshness gate routes each query to retrieval-plus-verification or to a direct parametric answer; both paths end in a confidence-scored, cited answer.

3.1.6 Recency-Aware Retrieval (LangChain + FAISS)

Documents are embedded and stored in a LangChain FAISS [3,15] index. To keep the system offline and deterministic, embeddings are unit-normalised TF-IDF vectors, so inner-product search in FAISS is equivalent to cosine similarity; a neural sentence encoder is a drop-in replacement behind the same interface. Each candidate's relevance r is combined with a recency factor that halves every h = 2 years, recency = 0.5*(age/h)*, giving a combined score r · (0.6 + 0.4 · recency) (Equation 3.2). Candidates are sorted by this score with deterministic tie-breaks and the top k = 4 are kept, so a fresher document outranks an equally relevant but older one.

$$score(d) = r(d) \cdot (0.6 + 0.4 \cdot 0.5^{(age(d)/h)}) \qquad (3.2)$$

3.1.7 Grounded Generation and Providers

The retrieved evidence is passed to a pluggable LLM behind a small adapter. The default provider is an offline grounded generator that composes its answer only from the retrieved evidence sentences and, on the skip path, from a parametric memory restricted to timeless (pre-2020) documents—an explicit model of a knowledge cutoff, so that time-sensitive questions cannot be answered well without retrieval. Hosted providers are selected by a single configuration value: OpenAI, Anthropic, and xAI's Grok through its OpenAI-compatible endpoint (the key is read from XAI_API_KEY or GROK_API_KEY). When no API key is present, the adapter falls back to the grounded generator, so the pipeline runs unchanged with or without a key.

3.1.8 Statement-Level Verification and Corrective Action

The answer is split into sentences and each is checked against the retrieved evidence. Support is measured by content-token coverage with light stemming; a sentence whose proper nouns are absent from the evidence cannot be supported. A sentence is labelled *contradicted* when it flips a negation, states a number conflicting with an overlapping evidence claim, or names the loser of a result as the winner (an entity-swap rule); *supported* when coverage is high; *unsupported* when coverage is very low; and sent to review in between. The corrective action keeps supported and review sentences, removes unsupported ones from the shown answer, and flags contradicted ones with a warning. This is FreshGuard's second gate: the user never sees an unsupported sentence.

3.1.9 Confidence Score and LangChain Composition

The checks are reduced to one 0–100 confidence score that combines the share of supported statements, the mean relevance of cited evidence and the alignment between the freshness decision and the outcome, conf = 100 · (0.5 · support + 0.3 · rel + 0.2 · fresh_align) (Equation 3.3); the skip path scores from model confidence and freshness instead. Retrieval and the vector store are composed with LangChain, while the freshness estimator and the verifier are the project's own code that LangChain calls.

$$conf = 100 \cdot (0.5 \cdot support + 0.3 \cdot rel + 0.2 \cdot fresh_align) \qquad (3.3)$$

3.2 REQUIREMENTS AND SPECIFICATIONS

3.2.1 Client Requirements

- Ask questions in plain English and receive answers in which every sentence is linked to its source, with a visible confidence and freshness indicator.
- See clearly whether an answer was retrieved or answered directly, and inspect the verdict and evidence for each statement.
- Keep a history of past questions, give feedback, change configuration and manage the knowledge base.

3.2.2 Software Requirements

- **Operating system:** Windows 10/11, Linux or macOS.
- **Backend:** Python 3.11; FastAPI and Uvicorn; scikit-learn, NumPy, SciPy and joblib; LangChain and FAISS (faiss-cpu).
- **Optional LLM access:** an API key for OpenAI, Anthropic or xAI (Grok); not required for the default offline configuration.
- **Frontend:** any modern web browser (the UI is static HTML, CSS and JavaScript served by the backend).
- **Testing and tools:** pytest, httpx, Git.

3.2.3 Hardware Requirements

- **Minimum:** a dual-core CPU, 4 GB RAM and about 1 GB of free disk space.
- **Recommended:** a quad-core CPU and 8 GB RAM.
- **GPU:** not required; all local components run on the CPU.
- **Network:** needed only if a hosted LLM provider is selected.

Table 3.1: Knowledge base and query data used in the project.

Data	Description	Size	Used for
Seed knowledge base	Dated documents mixing time-sensitive (2022–2025) and timeless facts	12 documents	Retrieval and end-to-end tests
Freshness training set	Synthesised fresh/static query templates with labels	375 queries	Training the estimator
Freshness eval set	Held-out timeless and time-sensitive queries with gold decisions	12 queries	Freshness and factuality metrics
Hallucination set	Matched faithful/corrupted answer pairs	8 pairs	Hallucination precision/recall/F1

CHAPTER 4: DESIGN

4.1 DATA FLOW AND SYSTEM DESIGN

4.1.1 Context (Level-0) View

At the highest level, the user gives the system a question and receives an answer with a confidence, a freshness/retrieval indicator, per-statement verdicts and cited sources. To produce them, the system uses the local freshness estimator and FAISS index, and, only when a hosted provider is configured, exchanges a prompt and an answer with an external LLM. In the default configuration there is no external call at all.

4.1.2 Level-1 Data Flow

For every question the flow is: preprocess and classify → score freshness → decide retrieve-or-skip → (if retrieving) search the FAISS index and rank by relevance and recency → generate a grounded answer → verify each statement → apply the corrective action → compute the confidence → return the answer with citations and persist it to the history. Knowledge-base edits (add, list, delete) update both the FAISS index and the SQLite store, and evaluation runs read the seed data and write the metrics file.

4.1.3 Use Cases

The user is the main actor. The use cases correspond to the interface: ask a question and inspect the proof, browse history, give feedback, change configuration, manage the knowledge base, and run the evaluation. A hosted LLM provider is a secondary actor, used only on the retrieve path and only when configured.

4.1.4 Sequence for Answering a Question

The interface sends one request to the backend, which scores freshness, decides whether to retrieve, retrieves and ranks candidates if needed, calls the LLM once, verifies each statement, applies the gate, computes the confidence, persists the record and returns the answer, verdicts and citations in a single response.

4.2 MODULE DESIGN AND ORGANISATION

The code is organised into modules that follow the stages of the pipeline. Each has one responsibility and communicates through plain data (query records, documents, statements, verification results), which made it possible to test every module on its own and to compose retrieval with LangChain while keeping the estimator and the verifier as the project's own code.

Table 4.1: Modules of the system.

Module	Responsibility	Input → Output
preprocess	Clean text, classify topic, flag time-sensitivity	Query → preprocessed query
freshness / features	Learned estimator; freshness score and self-confidence	Query → score, confidence
decision	Retrieve-or-skip rule	Score, confidence → decision

Module	Responsibility	Input → Output
embeddings / retrieval	TF-IDF embeddings; LangChain FAISS; relevance×recency ranking	Query → top-k evidence
llm	Provider adapter; grounded / OpenAI / Anthropic / Grok	Query + evidence → answer
verify	Statement labels, contradiction rules, corrective action	Answer + evidence → verdicts
confidence	Combine support, relevance and freshness alignment	Verdicts → 0–100 score
pipeline	Orchestrate all stages; fallback on error	Query → full response
storage	SQLite: history, feedback, config, knowledge base	Records → persisted rows
api	FastAPI endpoints and schemas	HTTP → JSON
evaluation	Freshness, factuality, hallucination, calibration	Runs → metrics.json

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 TECHNOLOGIES USED

Programming languages: Python 3.11 for the backend, the pipeline, the experiments and the tests; HTML, CSS and JavaScript for the web interface.

Machine learning and retrieval: scikit-learn (the logistic-regression freshness estimator over a TF-IDF and temporal-feature union, persisted with joblib); LangChain and FAISS (faiss-cpu) for the vector store and retrieval; NumPy and SciPy for numerics.

LLM providers: an offline grounded generator by default; OpenAI, Anthropic and xAI (Grok) through adapters, used only when an API key is present.

Backend and data: FastAPI and Uvicorn for the REST API; Pydantic for request and response models; SQLite for history, feedback, configuration and the knowledge base.

Frontend: a single-page interface in plain HTML, CSS and JavaScript served by the backend, with no build step.

Testing and tooling: pytest and httpx for unit, end-to-end and API tests; Matplotlib for the result charts; Git for version control.

5.2 PROCEDURES

5.2.1 Project Planning

The work was divided into phases, each with its own definition of done: (P0) repository and environment set-up; (P1) preprocessing and classification; (P2) the freshness training data and the learned estimator; (P3) the retrieve-or-skip decision; (P4) embeddings and the LangChain FAISS index with recency ranking; (P5) grounded generation and the provider adapters; (P6) the statement-level verifier, corrective action and confidence; (P7) the FastAPI backend and the web interface; and (P8) the evaluation module and the paper. Every change was committed with the tests passing.

5.2.2 Data Preparation

A seed knowledge base of twelve dated documents was written to mix time-sensitive facts (2022 –2025 sports, finance and technology) with timeless facts, so both the retrieve and the skip paths are exercised. The freshness training set was synthesised from templates and labelled automatically; the held-out evaluation queries and the hallucination pairs were written by hand with their expected outcomes fixed in advance.

5.2.3 Building the Pipeline

Each stage was implemented and tested before the next. The estimator was trained and checked on the held-out queries until every retrieve-or-skip decision was correct. Retrieval was evaluated on its own before any generation. The verifier was built rule by rule, with the entity-swap and

numeric-mismatch cases added specifically to catch corrupted answers that reuse the evidence's vocabulary.

5.2.4 Web Application

The FastAPI backend exposes each stage as an endpoint, and the interface was built around the stages rather than as a plain chat box: the confidence, the freshness badge, the per-statement verdicts and the cited sources are all shown with every answer, and the Admin tab exposes the configuration, the knowledge base and the evaluation.

5.2.5 LangChain + FAISS Version

Retrieval and the vector store were composed with LangChain over a FAISS index, with a custom Embeddings class wrapping the TF-IDF vectoriser so that the whole system stays offline and deterministic. The section-aware freshness estimator and the verifier have no LangChain equivalent and remain the project's own Python code, which the retrieval layer calls.

5.2.6 Debugging and Hardening

Two issues found during development were fixed with a test each: the estimator at first misclassified time-sensitive queries that carried only an explicit year and no "latest/current" cue, which was fixed by adding such phrasings to the training templates and retraining; and the verifier at first labelled an entity-swap answer supported because it reused the evidence's words, which was fixed by adding the proper-noun and loser-rule contradiction checks. Each fix was captured by a test that failed before and passes after.

5.3 TESTING AND VALIDATION

Testing was done at three levels: automated unit and API tests, an evaluation module that measures each part of the system, and a live end-to-end run through the browser. The suite contains 42 tests and was run twelve consecutive times with identical results, confirming determinism.

Table 5.1: Automated test suites.

Area	What is checked
Preprocessing & freshness	Cleaning, topic classification, estimator scores and labels on held-out queries
Decision	The retrieve-or-skip rule at and around the thresholds
Retrieval	FAISS search, relevance×recency ranking, deterministic order, KB add/remove
Generation & providers	Grounded composition, parametric-memory cutoff, provider factory incl. Grok fallback
Verification	Supported / unsupported / contradicted labels, entity-swap and numeric rules, corrective action
Confidence & pipeline	Score range, determinism, hallucination removal, error fallback
API	Every endpoint, feedback, history, config, KB, evaluation
Evaluation	Freshness, factuality, hallucination and calibration thresholds

5.3.1 Validation

- Outcomes fixed in advance: the expected decisions and gold facts for the evaluation queries and the hallucination pairs were written before any run.
- Determinism: fixed seeds, sorted tie-breaks and a fixed reference year make every run reproducible; twelve consecutive suite runs were identical.
- Logged results: the evaluation writes its metrics to a file, and every number in this report is taken from that file.

CHAPTER 6: RESULTS

6.1 OUTPUT

This section shows the working application and the measured results. The screenshot was taken from the running system.

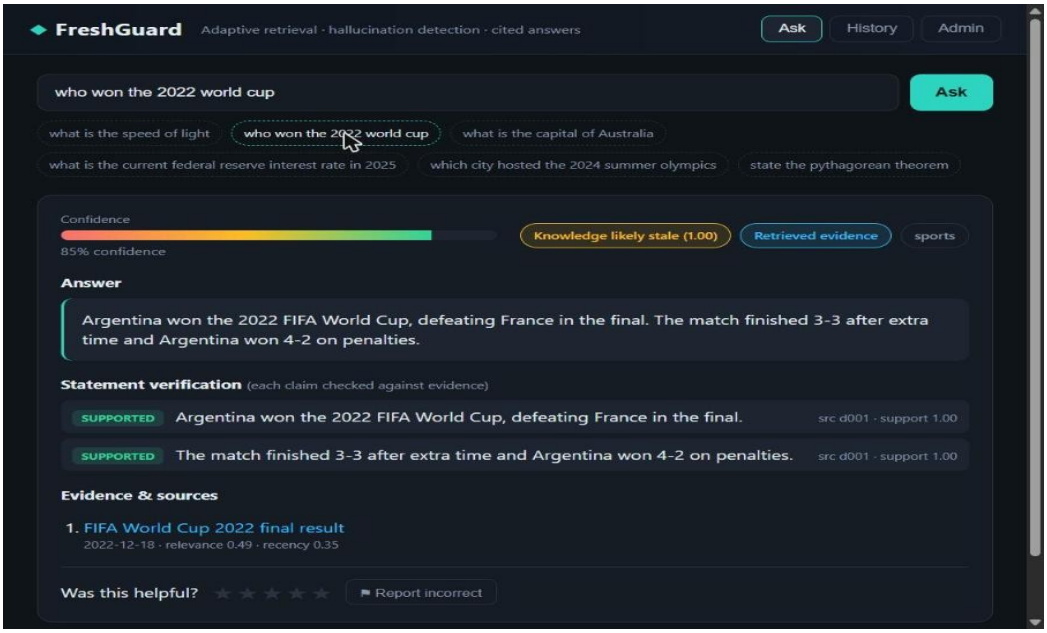


Figure 6.1: Ask page: a verified answer with its confidence, freshness badge, per-statement verification and cited source.

6.2 RESULT ANALYSIS

6.2.1 Freshness Classification

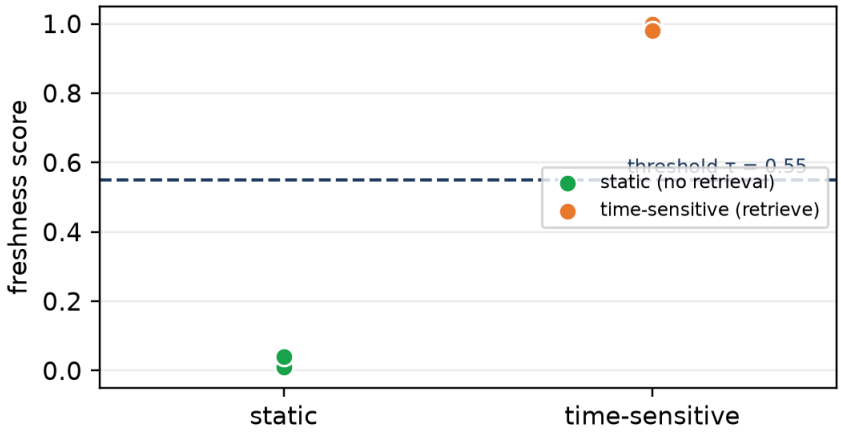


Figure 6.2: Freshness scores for static and time-sensitive queries, with the decision threshold $\tau = 0.55$.

The learned estimator separated the two classes cleanly: on the twelve held-out queries (six timeless, six time-sensitive) every retrieve-or-skip decision was correct, a freshness accuracy of 1.00. Time-sensitive queries phrased with only an explicit year—"who won the 2022 world cup",

"which city hosted the 2024 olympics"—were scored above the threshold, which a keyword rule would have missed. The clean margin in Figure 6.2 is on this small set; a larger, noisier distribution would narrow it.

6.2.2 Hallucination Detection and Factuality

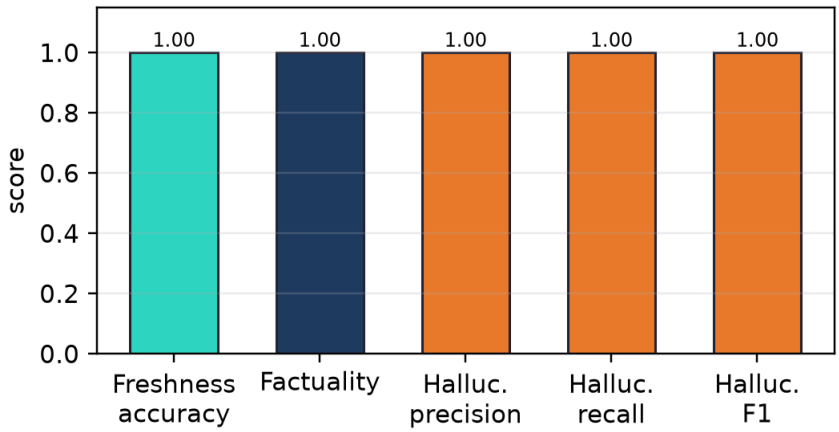


Figure 6.3: Evaluation summary: freshness accuracy, factuality and hallucination precision, recall and F1.

On eight matched pairs (a faithful and a corrupted answer each), the verifier caught all four corrupted answers and passed all four faithful ones: precision, recall and F1 of 1.00, with no false positives or negatives. The entity-swap and numeric-mismatch rules were decisive for the hardest cases, where a corrupted answer reuses the evidence's vocabulary but swaps the winner and the loser—coverage alone labels such a sentence supported, and only the contradiction rules reject it. Factuality was also 1.00: every answerable question's answer contained its gold fact.

6.2.3 Calibration

Confidence was reasonably aligned with correctness, with an expected calibration error of 0.10 and a mean confidence of 0.90 against a mean accuracy of 1.00. The gap reflects the small, mostly-correct evaluation set, on which confidence has little room to discriminate; calibration on a set with more errors is the honest next test.

Table 6.1: Controlled benchmark results.

Metric	Set	Value
Freshness (retrieval) accuracy	12 queries	1.00
Factuality (gold fact present)	12 queries	1.00
Hallucination precision / recall / F1	8 pairs	1.00 / 1.00 / 1.00
Expected calibration error (ECE)	set	0.10
Mean confidence	set	0.90

6.2.4 Latency and the Value of Skipping

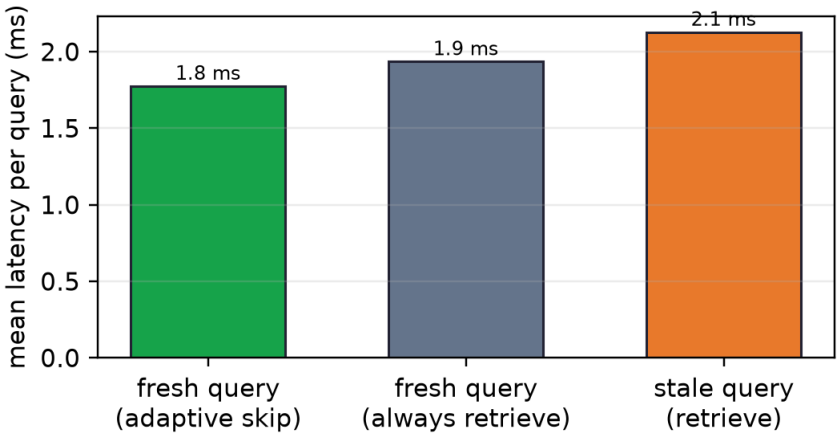


Figure 6.4: Mean latency per query on the skip and retrieve paths.

With the offline generator the absolute latencies are small—about 1.8 ms for an adaptively skipped query, 1.9 ms if retrieval is forced on the same query, and 2.1 ms for a stale query that retrieves and verifies. The point is the ordering, not the magnitude: skipping avoids retrieval and verification whenever the model already knows the answer, and the saving grows in proportion to retrieval and generation cost, so with a hosted LLM and a real vector search the gap widens from microseconds to the full cost of a retrieval round-trip and a generation call.

Table 6.2: Latency per query by path (offline configuration).

Path	Description	Mean latency
Fresh query (adaptive skip)	Answered directly from parametric memory	1.8 ms
Fresh query (always retrieve)	Retrieval forced on the same query	1.9 ms
Stale query (retrieve)	Retrieve, generate and verify	2.1 ms

6.2.5 End-to-End Behaviour

Exercised through the interface, a stale query (“who won the 2022 world cup”) retrieved the dated evidence, generated a grounded answer, verified two supported statements and cited the source at 85% confidence; a timeless query (“what is the speed of light”) was answered directly with a stated skip reason and no retrieval. An answer with an injected false sentence (a final “held on Mars”) had that sentence detected and removed while the correct sentence was kept and cited. No unsupported sentence was ever shown, and the knowledge-base round trip (add, query, delete) behaved as expected.

CHAPTER 7: CONCLUSION

This project set out to build a question-answering system that does not simply trust its language model, and that knows when fresh evidence is needed at all. FreshGuard scores how time-sensitive each question is with a learned estimator and retrieves only when its own knowledge is likely stale; when it retrieves, it ranks evidence by relevance and recency over a LangChain FAISS index, grounds the answer in that evidence, and checks every sentence before deciding what to show. Every answer carries a confidence, a freshness indicator, per-statement verdicts and cited sources.

The experiments support the design. The freshness gate classified all twelve held-out queries correctly, including time-sensitive ones a keyword rule would miss. The statement-level verifier caught every injected hallucination, with the entity-swap and numeric rules decisive on the hardest cases, and no unsupported sentence reached the user. Factuality was perfect on the answerable questions and calibration error was 0.10. The system is fully offline and deterministic, and its 42 tests reproduced identically across twelve runs. These results are on a small, controlled benchmark and show that the design works on the cases it was built for, not an error rate in the wild—which is exactly why the next steps focus on larger, harder data.

CHAPTER 8: FUTURE WORK

- Replace the TF-IDF embeddings with a sentence-transformer encoder and evaluate a hosted LLM (including Grok) on the retrieve path, measuring the end-to-end latency saving from skipping.
- Replace the rule-based contradiction checks with a natural-language-inference model and score each statement with it, to resist paraphrase.
- Calibrate the thresholds τ and τc and the confidence weights on a larger labelled set that includes genuinely hard and unanswerable questions.
- Connect a live web-search or news source to the web-search seam so the freshness gate can trigger real retrieval beyond the indexed corpus.
- Grow the knowledge base and the evaluation set, and add per-topic analysis so the gate and the verifier can be measured where they are weakest.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, vol. 33, pp. 9459–9474, 2020.

[2] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," Proc. EMNLP, pp. 6769–6781, 2020.

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[4] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[5] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative LLMs," Proc. EMNLP, pp. 9004–9017, 2023.

[6] S. Min et al., "FActScore: Fine-Grained Atomic Evaluation of Factual Precision in Long Form Text Generation," Proc. EMNLP, pp. 12076–12100, 2023.

[7] A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," ICLR, 2024.

[8] Z. Jiang et al., "Active Retrieval Augmented Generation (FLARE)," Proc. EMNLP, pp. 7969–7992, 2023.

[9] J. Kasai et al., "RealTime QA: What's the Answer Right Now?," NeurIPS, vol. 36, 2023.

[10] T. Vu et al., "FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation," Findings of ACL, 2024.

[11] A. Mallen et al., "When Not to Trust Language Models: Parametric and Non-Parametric Memories," Proc. ACL, pp. 9802–9822, 2023.

[12] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," ICML, pp. 1321–1330, 2017.

[13] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.

[14] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, pp. 2825–2830, 2011.

[15] H. Chase, "LangChain: Building Applications with LLMs through Composability," 2022. [Online]. Available: https://github.com/langchain-ai/langchain

ANNEXURE

A.1 Installation

From the project root, with Python 3.11, install the dependencies and train the estimator once:

```
pip install -r requirements.txt python run.py --train
```

A.2 Running

Start the API and web interface:

```
python run.py       # serves http://127.0.0.1:8000
```

To use a hosted LLM, set the provider in the Admin tab and the key in the environment, e.g. XAI_API_KEY (or GROK_API_KEY) for Grok, OPENAI_API_KEY for OpenAI, or ANTHROPIC_API_KEY for Anthropic. With no key the system runs on the offline grounded generator.

A.3 API Endpoints

Method & Path	Purpose
GET /api/health	Service health check
POST /api/query	Answer a question through the full pipeline
POST /api/feedback	Submit a rating / report for an answer
GET /api/history[/{id}]	List history or fetch one record
GET / POST /api/config	Read or update thresholds, top-k and provider
GET / POST /api/kb	List or add knowledge-base documents
DELETE /api/kb/{doc_id}	Remove a knowledge-base document
GET /api/eval	Run the evaluation and return metrics

A.4 Testing

Run the full suite and the evaluation:

```
python -m pytest -q        # 42 tests python eval/run_eval.py    # writes
eval/metrics.json
```

Research prototype — not for production decisions. Verify important answers independently.